DATABRICKS FREE EDITION

Parquet & PySpark Hands-on Practice Workbook

1. Lab overview

You are a junior data engineer at a retail/e-commerce company. Your task is to ingest source data, convert it to Parquet, explore it with Spark SQL and PySpark, perform joins and aggregations, and publish a small analytical Gold dataset.

Learning objectives

- Read CSV files into Spark DataFrames and inspect schemas.
- Write and read Parquet datasets.
- Register temporary views and query them with SQL.
- Clean missing values and check data quality.
- Join datasets and calculate business metrics.
- Use window functions to rank products and find latest orders.
- Write analytical outputs as Parquet and Delta tables.

2. Dataset inventory

File	Rows	Description
customers.csv	120	Customer name, email, city, region, segment and signup date
products.csv	21	Product name, category, unit cost and list price
orders.csv	1,500	Order date, customer/product keys, quantity, price, discount, status and payment

3. Upload files to Databricks

1. Open your Databricks Free Edition workspace.
2. Open Catalog and select your workspace catalog (often named workspace).
3. Create a schema named retail_lab if it does not already exist.
4. Create a volume named raw_files inside retail_lab.
5. Upload customers.csv, products.csv and orders.csv into that volume.
6. If your catalog has a different name, replace workspace in the paths below.

Expected file paths:

```
/Volumes/workspace/retail_lab/raw_files/customers.csv
/Volumes/workspace/retail_lab/raw_files/products.csv
/Volumes/workspace/retail_lab/raw_files/orders.csv
```

4. Read the CSV datasets

Create a Python notebook and run:

```
catalog = "workspace"
schema = "retail_lab"
base = f"/Volumes/{catalog}/{schema}/raw_files"

customers = (spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(f"{base}/customers.csv"))

products = (spark.read
```

```
    .option("header", True)
    .option("inferSchema", True)
    .csv(f"{base}/products.csv"))

orders = (spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(f"{base}/orders.csv"))

display(customers.limit(5))
display(products.limit(5))
display(orders.limit(5))
```

5. Convert CSV to Parquet

Parquet is a columnar file format commonly used for analytical workloads.

```
parquet_base = f"/Volumes/{catalog}/{schema}/parquet"

customers.write.mode("overwrite").parquet(f"{parquet_base}/customers")
products.write.mode("overwrite").parquet(f"{parquet_base}/products")
orders.write.mode("overwrite").parquet(f"{parquet_base}/orders")

# Read the Parquet data back
orders_parquet = spark.read.parquet(f"{parquet_base}/orders")
orders_parquet.printSchema()
display(orders_parquet.limit(10))
```

The downloadable data pack supplies CSV source files. Run the conversion code in Databricks to create the actual Parquet datasets in your workspace.

6. Register temporary views for SQL

```
customers_parquet = spark.read.parquet(f"{parquet_base}/customers")
products_parquet = spark.read.parquet(f"{parquet_base}/products")
orders_parquet = spark.read.parquet(f"{parquet_base}/orders")

customers_parquet.createOrReplaceTempView("customers")
products_parquet.createOrReplaceTempView("products")
orders_parquet.createOrReplaceTempView("orders")
```

You can now run SQL in a %sql cell or through spark.sql(...).

7. Practical questions

Level 1 – Basics

7. Display the first 10 orders.
8. Show the schema and row count of each dataset.
9. Find all orders with status Delivered.
10. Show orders placed in 2025.
11. List all distinct order statuses.
12. Find orders with a missing shipping_date.

Level 2 – Aggregations

13. Calculate the total number of orders by status.
14. Calculate total quantity sold by product.

15. Calculate revenue by payment method. Revenue = quantity × unit_price × (1 − discount_pct). Treat a missing discount as 0.
16. Find monthly revenue for 2025.
17. Find the top 5 customers by total order value.
18. Calculate average order value by customer segment.

Level 3 – Joins and data quality

19. Join orders to customers and show order ID, customer name, city and region.
20. Join orders to products and calculate revenue by product category.
21. Find customers who have never placed an order.
22. Identify duplicate order IDs.
23. Replace missing discounts with 0 and verify there are no NULL discounts.
24. Calculate the percentage of orders in each status.

Level 4 – Window functions and storage

25. Rank products by revenue within each category.
26. For each customer, find their most recent order using ROW_NUMBER.
27. Create a Gold table containing monthly revenue by region.
28. Save the Gold table as Parquet and read it back.
29. Save cleaned orders as a Delta table and query it with SQL.
30. Explain what additional capabilities Delta provides compared with Parquet.

8. SQL solutions

Run these after registering the customers, products and orders temporary views. The revenue calculation uses COALESCE(discount_pct, 0), so missing discounts are treated as zero.

```
-- Run after registering the three temp views: customers, products, orders.
-- Revenue definition: quantity * unit_price * (1 - coalesce(discount_pct, 0))

-- 1. First 10 orders
SELECT * FROM orders LIMIT 10;

-- 2. Row counts
SELECT 'customers' AS dataset, COUNT(*) AS rows FROM customers
UNION ALL SELECT 'products', COUNT(*) FROM products
UNION ALL SELECT 'orders', COUNT(*) FROM orders;

-- 3. Delivered orders
SELECT * FROM orders WHERE order_status = 'Delivered';

-- 4. Orders placed in 2025
SELECT * FROM orders WHERE YEAR(TO_DATE(order_date)) = 2025;

-- 5. Distinct statuses
SELECT DISTINCT order_status FROM orders ORDER BY order_status;

-- 6. Missing shipping date
SELECT * FROM orders WHERE shipping_date IS NULL;

-- 7. Orders by status
SELECT order_status, COUNT(*) AS order_count
FROM orders GROUP BY order_status ORDER BY order_count DESC;

-- 8. Quantity sold by product
SELECT p.product_name, SUM(o.quantity) AS units_sold
FROM orders o JOIN products p USING (product_id)
GROUP BY p.product_name ORDER BY units_sold DESC;

-- 9. Revenue by payment method
SELECT payment_method,
       ROUND(SUM(quantity * unit_price * (1 - COALESCE(discount_pct, 0))), 2) AS revenue
FROM orders GROUP BY payment_method ORDER BY revenue DESC;

-- 10. Monthly revenue in 2025
SELECT DATE_FORMAT(TO_DATE(order_date), 'yyyy-MM') AS order_month,
       ROUND(SUM(quantity * unit_price * (1 - COALESCE(discount_pct, 0))), 2) AS revenue
FROM orders
WHERE YEAR(TO_DATE(order_date)) = 2025
GROUP BY DATE_FORMAT(TO_DATE(order_date), 'yyyy-MM')
ORDER BY order_month;

-- 11. Top 5 customers by order value
SELECT c.customer_id, c.customer_name,
       ROUND(SUM(o.quantity * o.unit_price * (1 - COALESCE(o.discount_pct, 0))), 2) AS total_value
FROM orders o JOIN customers c USING (customer_id)
GROUP BY c.customer_id, c.customer_name
ORDER BY total_value DESC LIMIT 5;

-- 12. Average order value by customer segment
WITH order_values AS (
  SELECT o.order_id, c.customer_segment,
         SUM(o.quantity * o.unit_price * (1 - COALESCE(o.discount_pct, 0))) AS order_value
  FROM orders o JOIN customers c USING (customer_id)
  GROUP BY o.order_id, c.customer_segment
)
SELECT customer_segment, ROUND(AVG(order_value), 2) AS avg_order_value
FROM order_values GROUP BY customer_segment;
```

```
-- 13. Orders with customer details
SELECT o.order_id, c.customer_name, c.city, c.region
FROM orders o JOIN customers c USING (customer_id);

-- 14. Revenue by product category
SELECT p.category,
       ROUND(SUM(o.quantity * o.unit_price * (1 - COALESCE(o.discount_pct, 0))), 2) AS revenue
FROM orders o JOIN products p USING (product_id)
GROUP BY p.category ORDER BY revenue DESC;

-- 15. Customers without orders
SELECT c.*
FROM customers c LEFT ANTI JOIN orders o ON c.customer_id = o.customer_id;

-- 16. Duplicate order IDs
SELECT order_id, COUNT(*) AS occurrences
FROM orders GROUP BY order_id HAVING COUNT(*) > 1;

-- 17. Missing discounts after replacement
SELECT COUNT(*) AS null_discount_count
FROM (SELECT COALESCE(discount_pct, 0) AS discount_pct FROM orders)
WHERE discount_pct IS NULL;

-- 18. Percentage of orders by status
SELECT order_status, COUNT(*) AS order_count,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_orders
FROM orders GROUP BY order_status ORDER BY pct_of_orders DESC;

-- 19. Rank products by revenue within category
WITH product_revenue AS (
  SELECT p.category, p.product_id, p.product_name,
         SUM(o.quantity * o.unit_price * (1 - COALESCE(o.discount_pct, 0))) AS revenue
  FROM products p JOIN orders o USING (product_id)
  GROUP BY p.category, p.product_id, p.product_name
)
SELECT *, DENSE_RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS category_rank
FROM product_revenue;

-- 20. Most recent order per customer
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY TO_DATE(order_date) DESC, order_id DESC) AS
rn
  FROM orders
)
SELECT * FROM ranked WHERE rn = 1;

-- 21. Gold aggregate: monthly revenue by region
CREATE OR REPLACE TABLE monthly_revenue_by_region AS
SELECT DATE_FORMAT(TO_DATE(o.order_date), 'yyyy-MM') AS order_month,
       c.region,
       ROUND(SUM(o.quantity * o.unit_price * (1 - COALESCE(o.discount_pct, 0))), 2) AS revenue
FROM orders o JOIN customers c USING (customer_id)
GROUP BY DATE_FORMAT(TO_DATE(o.order_date), 'yyyy-MM'), c.region;

-- 22. Write/read Parquet (PySpark cell)
-- spark.table("monthly_revenue_by_region").write.mode("overwrite").parquet(
--     f"/Volumes/{catalog}/{schema}/parquet/monthly_revenue_by_region")
-- spark.read.parquet(f"/Volumes/{catalog}/{schema}/parquet/monthly_revenue_by_region").show()

-- 23. Cleaned Delta table (PySpark cell)
-- cleaned_orders = orders_parquet.fillna({"discount_pct": 0})
-- cleaned_orders.write.format("delta").mode("overwrite").saveAsTable("retail_lab.cleaned_orders")
```

9. PySpark solution examples

```
# PySpark answer snippets. Assumes customers, products, orders are DataFrames.
from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Basic counts and delivered orders
for name, df in [("customers", customers), ("products", products), ("orders", orders)]:
    print(name, df.count())
orders.filter(F.col("order_status") == "Delivered").show()

# Create a reusable revenue column; null discount is treated as zero
orders_clean = orders.fillna({"discount_pct": 0.0})
orders_clean = orders_clean.withColumn(
    "revenue",
    F.col("quantity") * F.col("unit_price") * (1 - F.col("discount_pct"))
)

# Revenue by payment method
orders_clean.groupBy("payment_method").agg(
    F.round(F.sum("revenue"), 2).alias("revenue")
).orderBy(F.desc("revenue")).show()

# Join orders with customers
orders_with_customers = orders_clean.join(customers, "customer_id", "inner")
orders_with_customers.select("order_id", "customer_name", "city", "region").show()

# Revenue by product category
orders_with_products = orders_clean.join(products, "product_id", "inner")
orders_with_products.groupBy("category").agg(
    F.round(F.sum("revenue"), 2).alias("revenue")
).orderBy(F.desc("revenue")).show()

# Customers who have never ordered
customers.join(
    orders.select("customer_id").distinct(),
    "customer_id",
    "left_anti"
).show()

# Most recent order per customer
w = Window.partitionBy("customer_id").orderBy(
    F.to_date("order_date").desc(), F.col("order_id").desc()
)
latest_orders = orders.withColumn("rn", F.row_number().over(w)).filter("rn = 1")
latest_orders.show()

# Monthly revenue by region
monthly_gold = (
    orders_clean.join(customers, "customer_id")
    .withColumn("order_month", F.date_format(F.to_date("order_date"), "yyyy-MM"))
    .groupBy("order_month", "region")
    .agg(F.round(F.sum("revenue"), 2).alias("revenue"))
)
monthly_gold.write.mode("overwrite").parquet(
    f"{parquet_base}/monthly_revenue_by_region"
)

# Write cleaned data as a managed Delta table
orders_clean.write.format("delta").mode("overwrite").saveAsTable(
    "retail_lab.cleaned_orders"
)
```

10. Data dictionary

DATA DICTIONARY

customers.csv

- - customer_id: unique customer key
- - customer_name: generated customer name
- - email: example email address
- - city, region: customer geography
- - signup_date: customer registration date
- - customer_segment: Consumer, Corporate, or Small Business

products.csv

- - product_id: unique product key
- - product_name: product description
- - category: Electronics, Furniture, Stationery, Accessories
- - unit_cost: illustrative cost per unit
- - list_price: illustrative selling price per unit

orders.csv

- - order_id: unique order key
- - order_date: order date
- - customer_id: customer foreign key
- - product_id: product foreign key
- - quantity: units ordered
- - unit_price: selling price per unit
- - discount_pct: discount fraction (e.g. 0.10 = 10%); some values are null
- - order_status: Delivered, Shipped, Pending, Cancelled, Returned
- - shipping_date: ship date; some values are null
- - payment_method: payment channel

NOTE: All records and names are synthetic and intended for learning only.